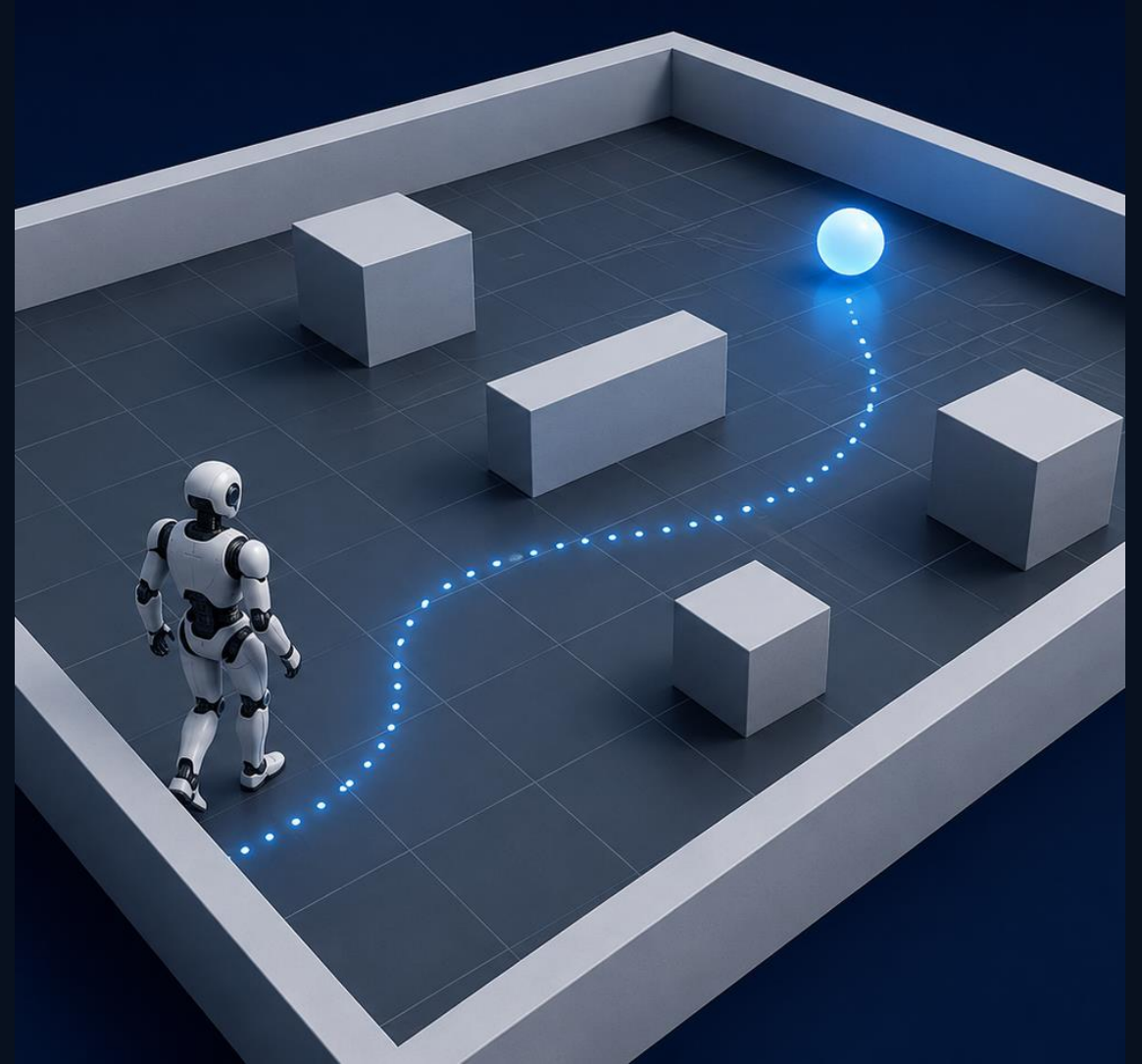


TALENTUD-IK PROJECT

Reinforcement Learning- Based Robot Navigation

Unity ML-Agents · PPO · Behavioral Cloning · GAIL ·
RayPerceptionSensor3D



Azish Ahmed · 2025

Project Snapshot

Autonomous navigation in a Unity simulation environment

PPO

main RL algorithm

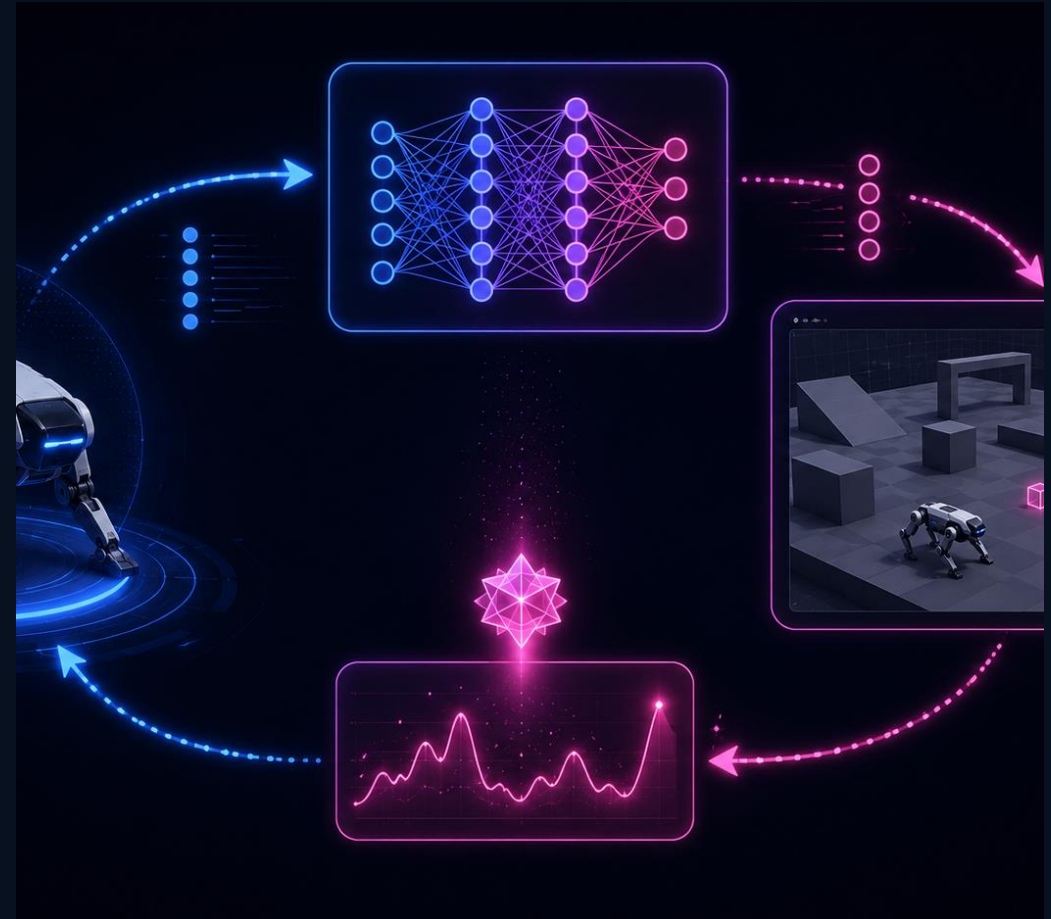
BC + GAIL

demonstration-guided learning

LiDAR-like

ray-based sensing

Goal: train a humanoid robot to reach a random target while avoiding walls and obstacles.



Technologies Used

Simulation, training and result analysis stack

Unity 2022.3 LTS

simulation engine

mlagents 1.1.0

Python trainer

TensorBoard

training graphs

ML-Agents 2.0.2

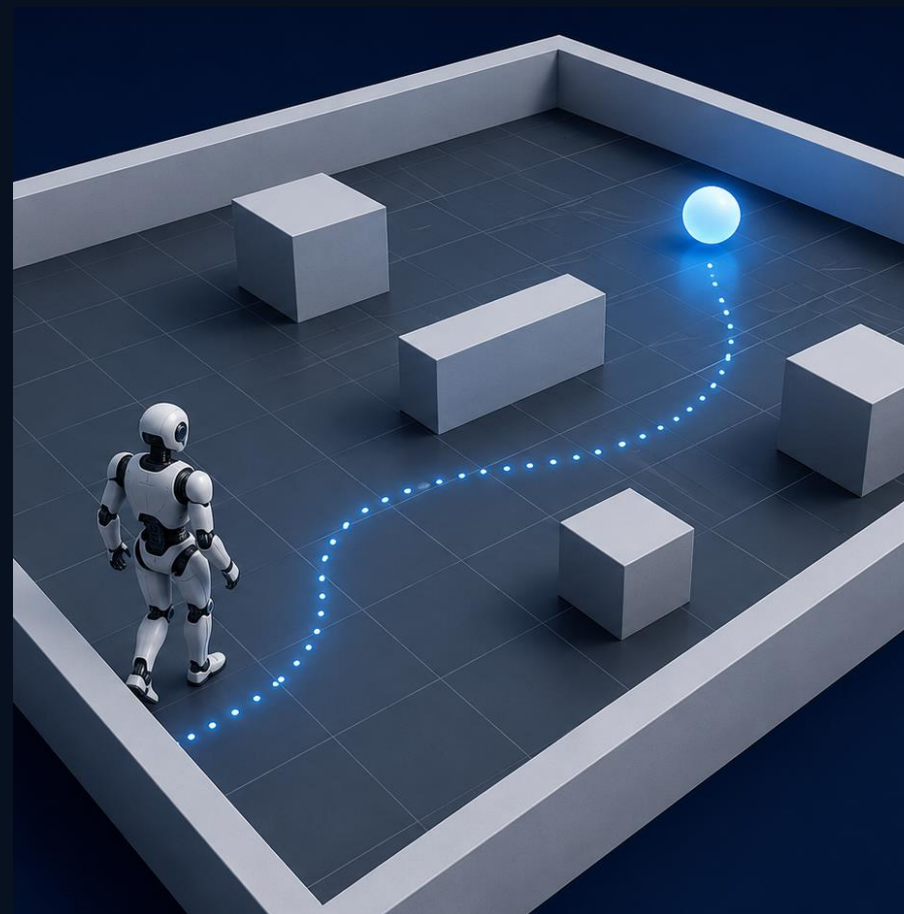
agent components

PyTorch CPU

neural networks

Mixamo humanoid

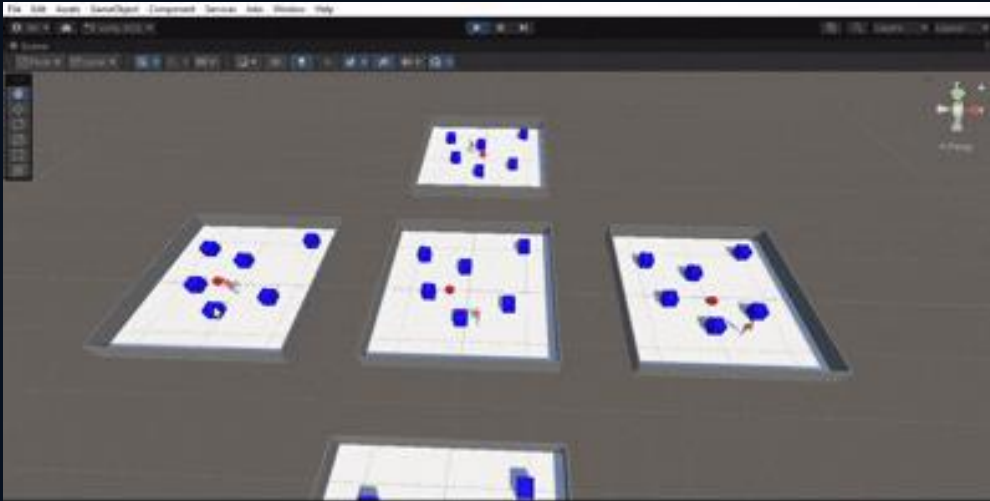
robot model



Training was designed for a low-resource CPU-only laptop, so vector and ray sensors were prioritized over camera-based observations.

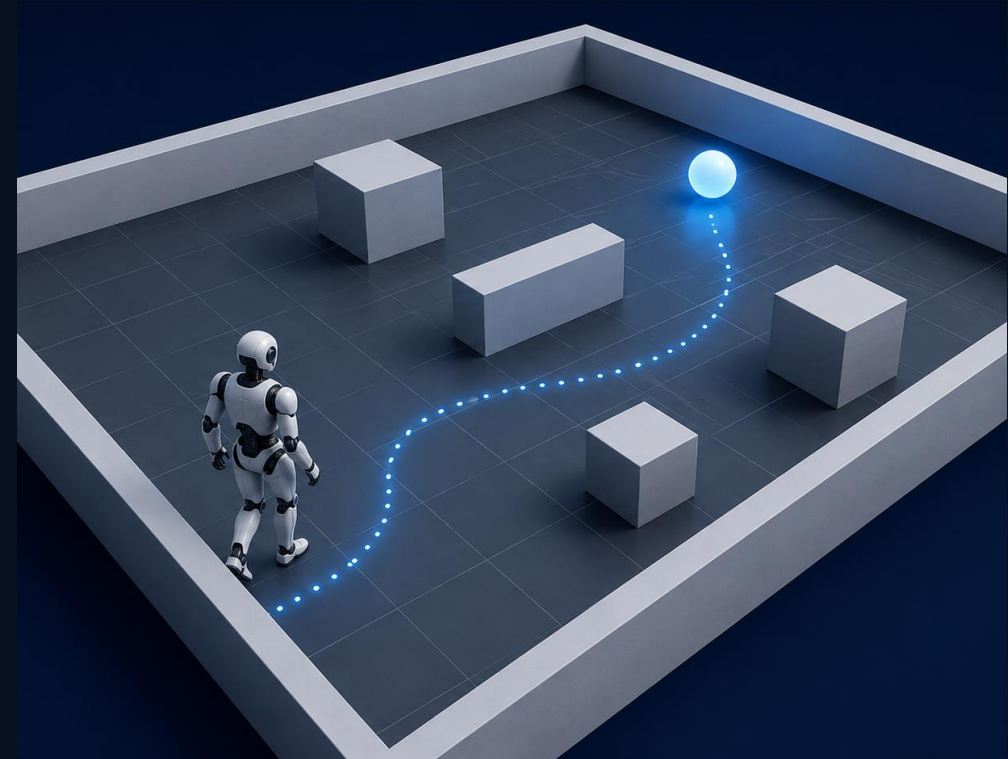
Unity Environment

Multiple training arenas with randomized target navigation



Scene elements

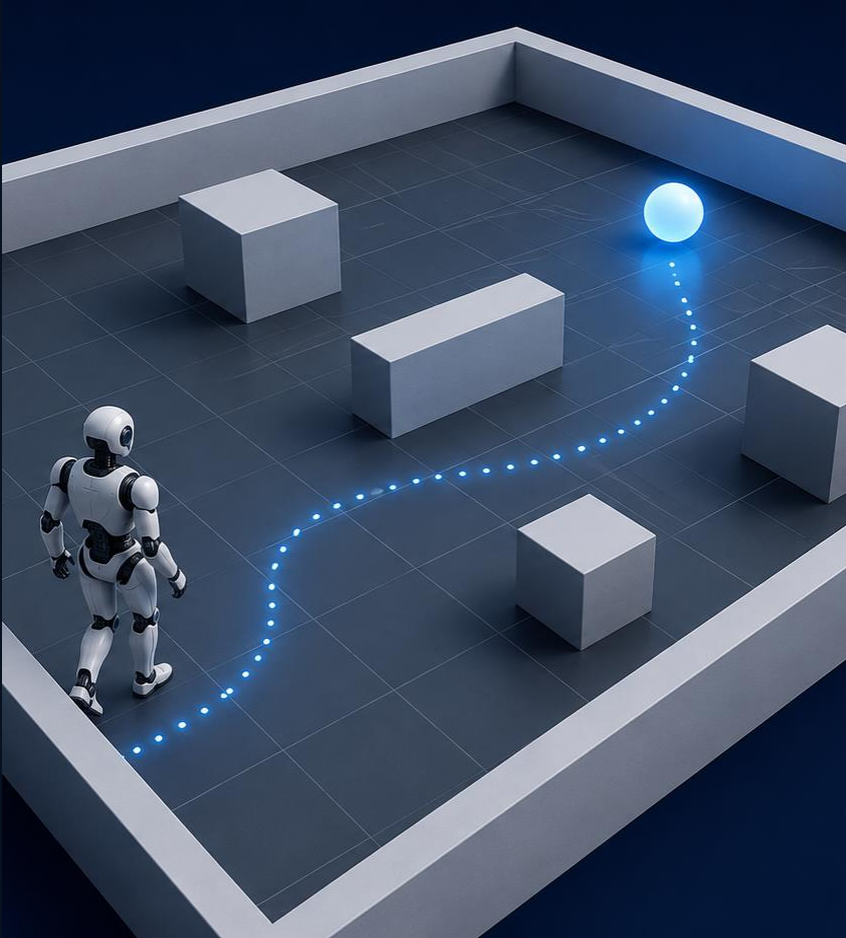
- floor and boundary walls
- obstacle objects
- random target sphere
- color feedback for success/failure



The target changes position between episodes so the policy learns general navigation instead of memorizing one path.

RobotAgent Architecture

Clean separation between physics, control and animation



Parent
RobotAgent

Rigidbody
Collider

Child FBX
Humanoid mesh

Animator
speed parameter

Continuous actions
move + rotate

Using an empty parent object fixed movement and animation issues by keeping RL control separate from the visual Mixamo character.

Observations and Actions

What the policy receives and controls

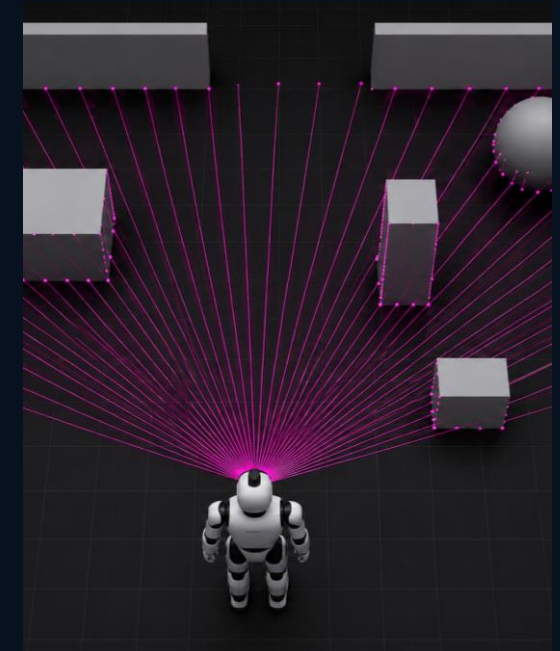
Action space

Action 0: forward / backward movement

Action 1: left / right rotation

Vector observations

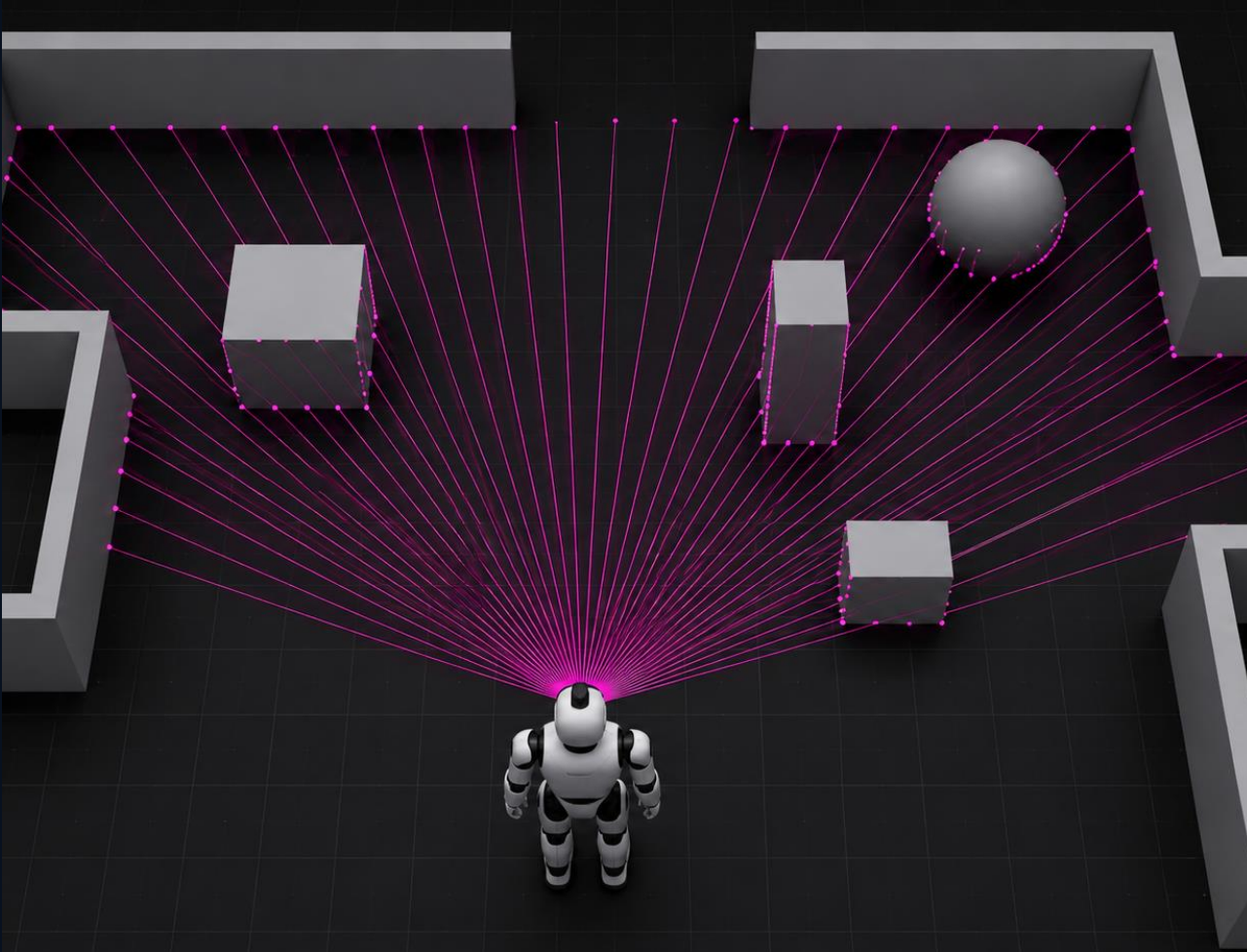
target direction	3
target distance	1
local velocity	3
forward direction	3
speed magnitude	1
walking speed	1



RayPerceptionSensor3D
adds obstacle and target
detection around the robot.

LiDAR-Style Perception

RayPerceptionSensor3D simulates range-based sensing



Sensor setup

8

rays per direction

90°

detection angle

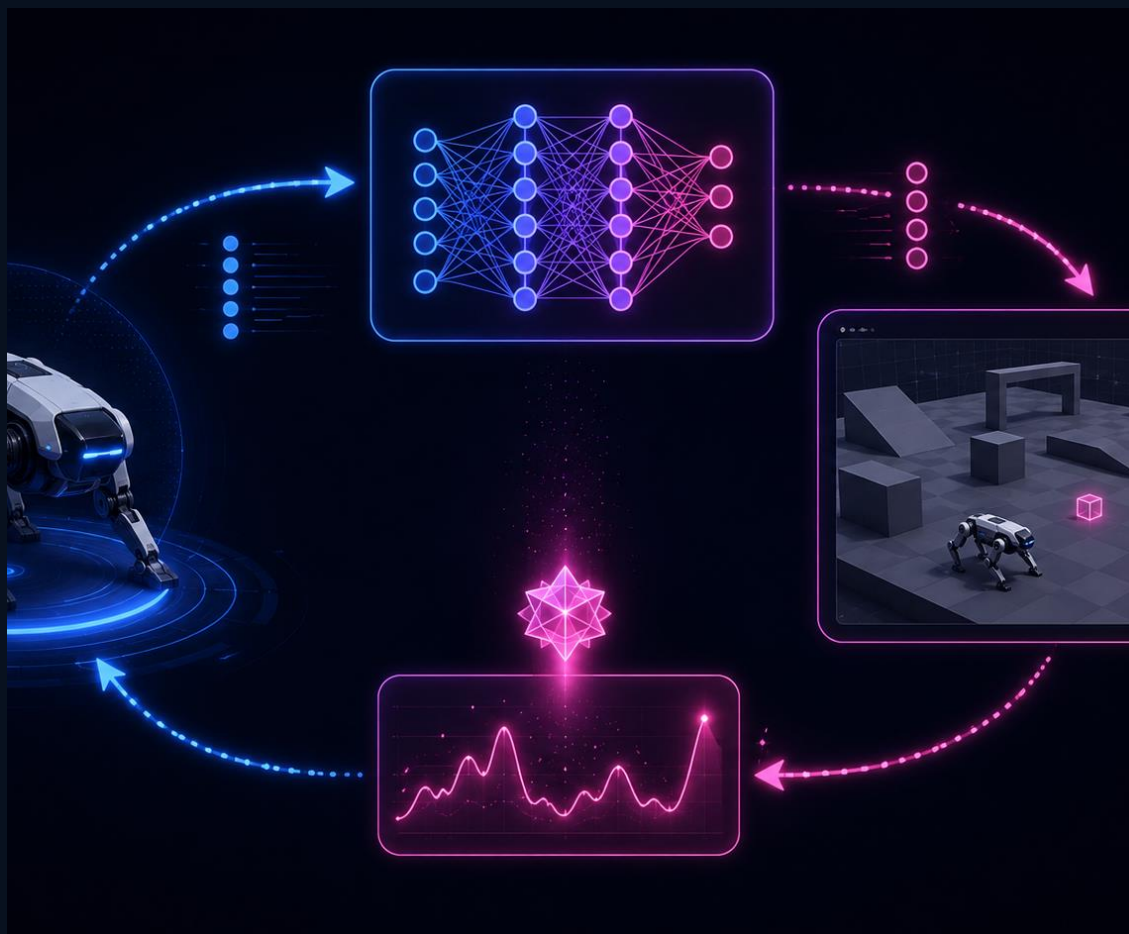
3 tags

Wall / Obstacle / Target

This sensor made the task more realistic than relying only on target coordinates.

Learning Pipeline

PPO trained with demonstration support



Manual demo

RobotNavDemo_v2

Behavioral Cloning

pretraining signal

GAIL

imitation reward

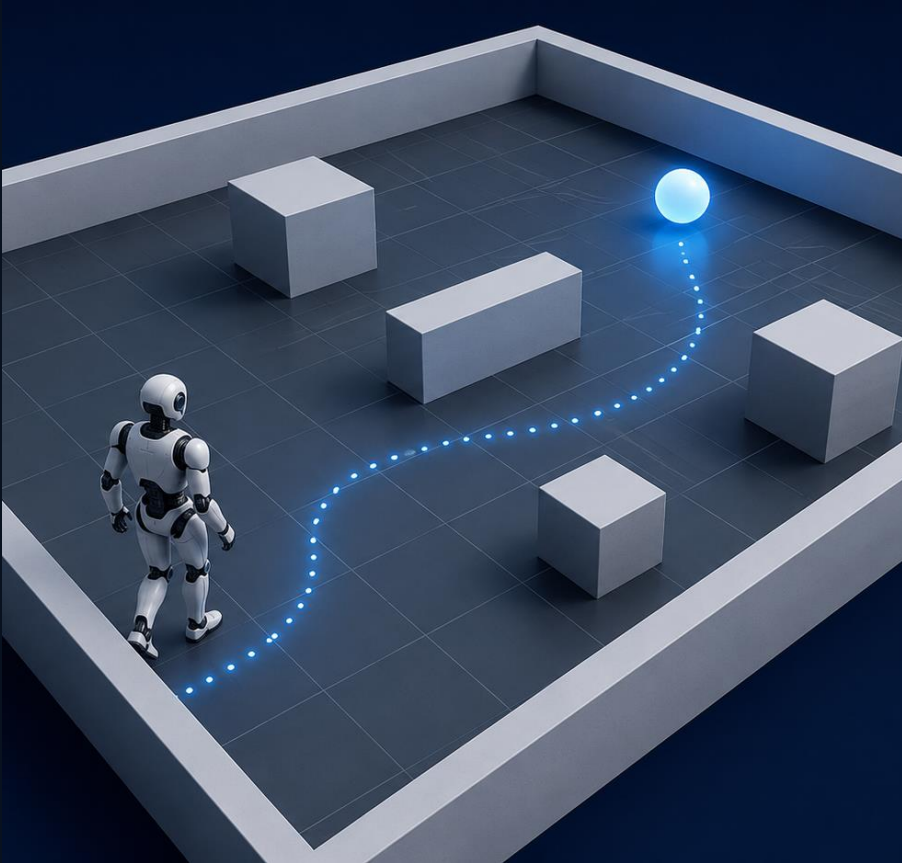
PPO

policy optimization

Demonstrations helped avoid fully random exploration at the beginning of training.

Reward Engineering

Removing reward exploitation and encouraging task completion



Original issue

The dense distance reward encouraged the agent to stay near the target instead of finishing the episode.

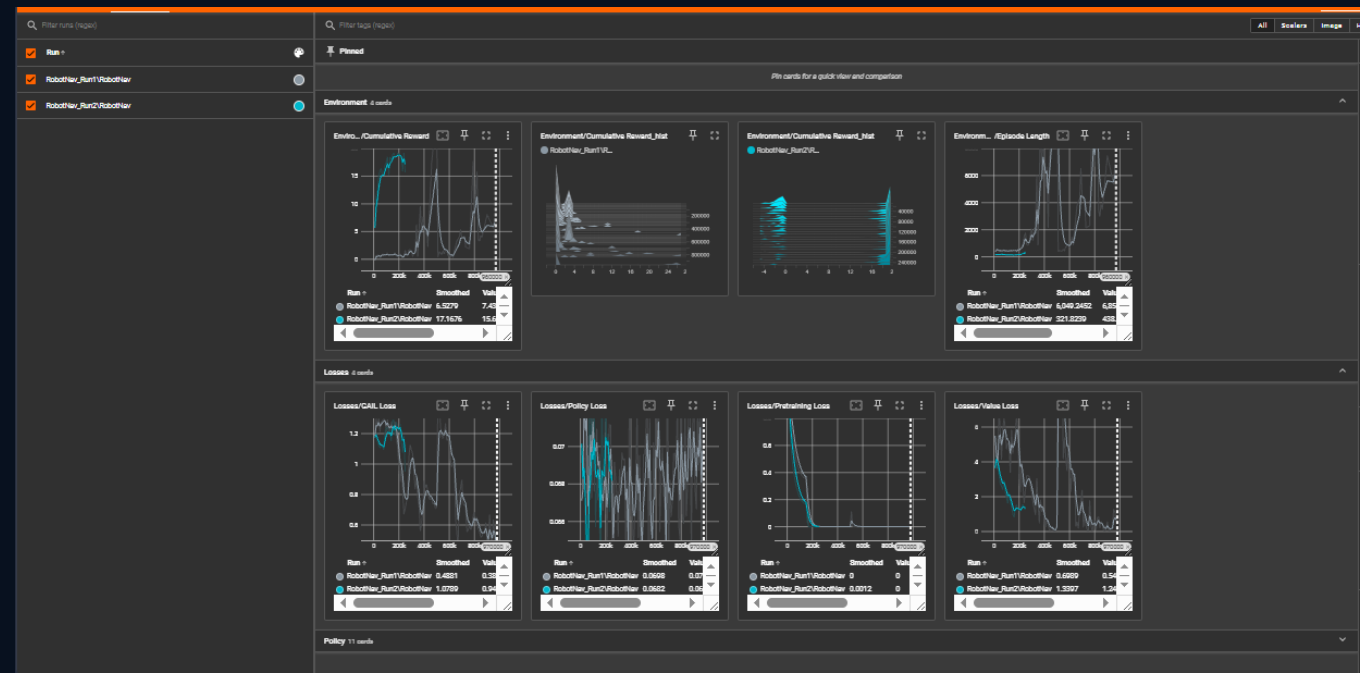
Final reward design

Target reached	+20.0
Collision	-0.5
Step penalty	-0.001
Orientation shaping	$\text{dot} \times 0.0005$

Result: the policy had a stronger incentive to complete the task.

Training Results

TensorBoard PPO training metrics



15-18

cumulative reward range after improvement



pretraining loss decreased

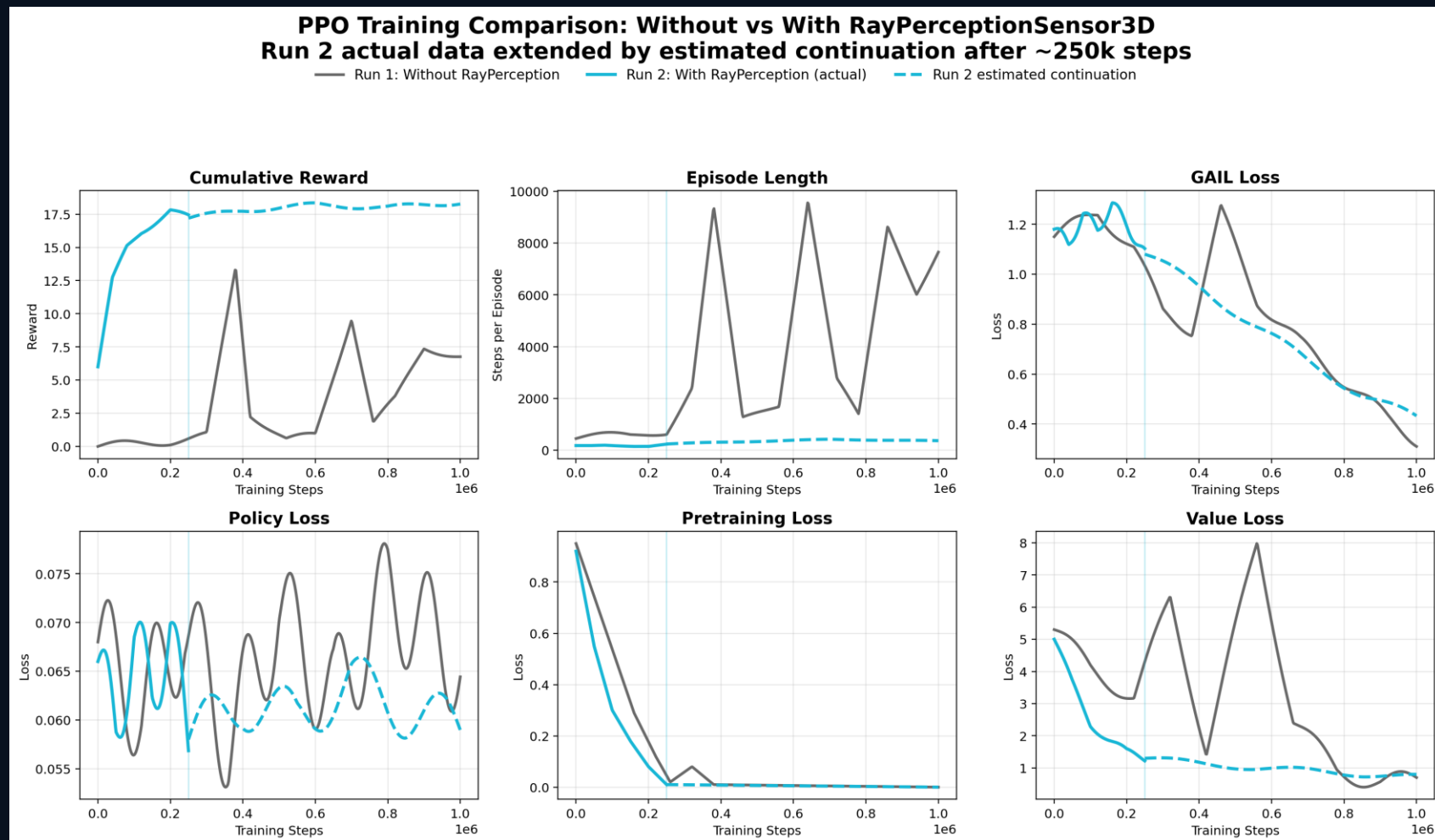
stable trend

value loss improved overall

The results show useful navigation learning after the reward exploit was removed.

Sensor Comparison

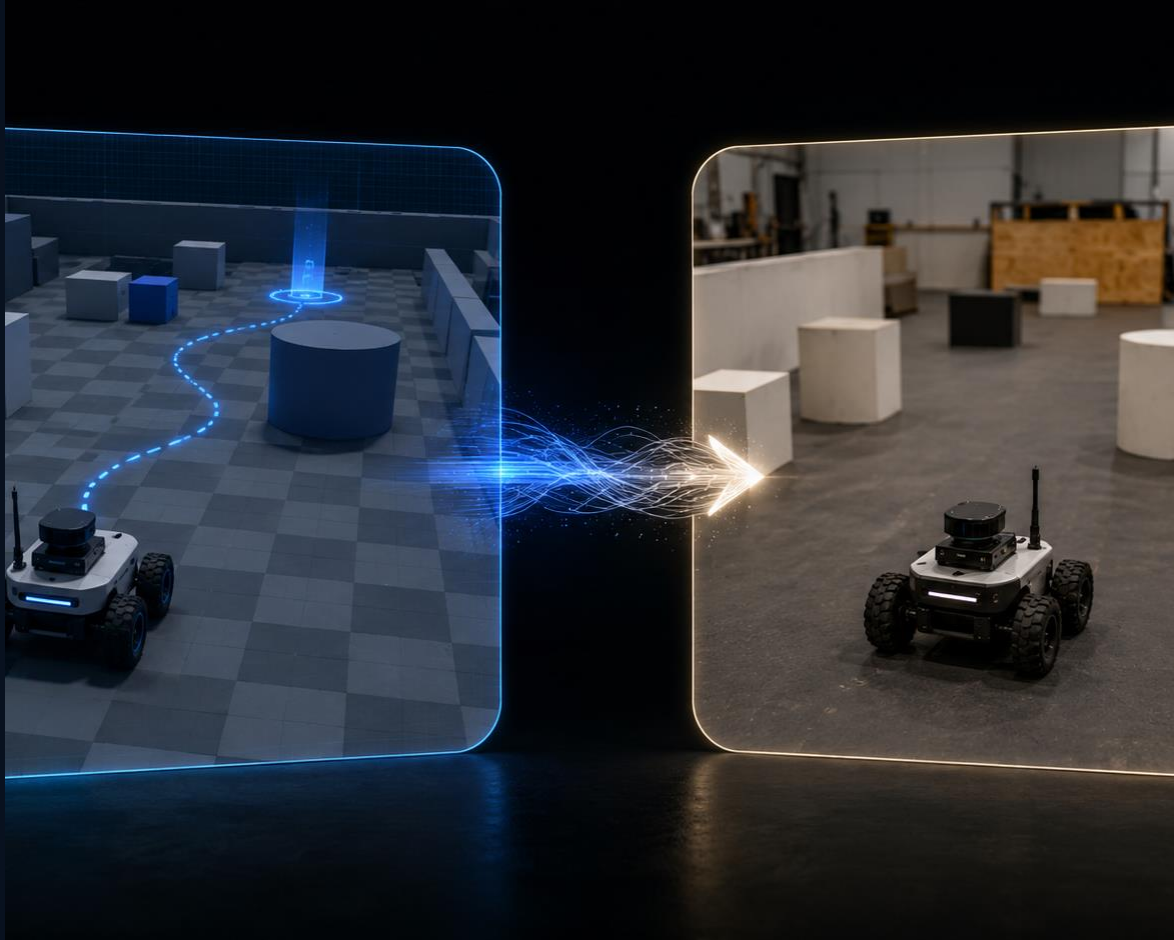
Run 1 without rays vs Run 2 with RayPerceptionSensor3D



Dashed cyan line represents an estimated continuation of the ray-sensor run beyond the available training steps.

Sim-to-Real Perspective

What would change for a physical robot

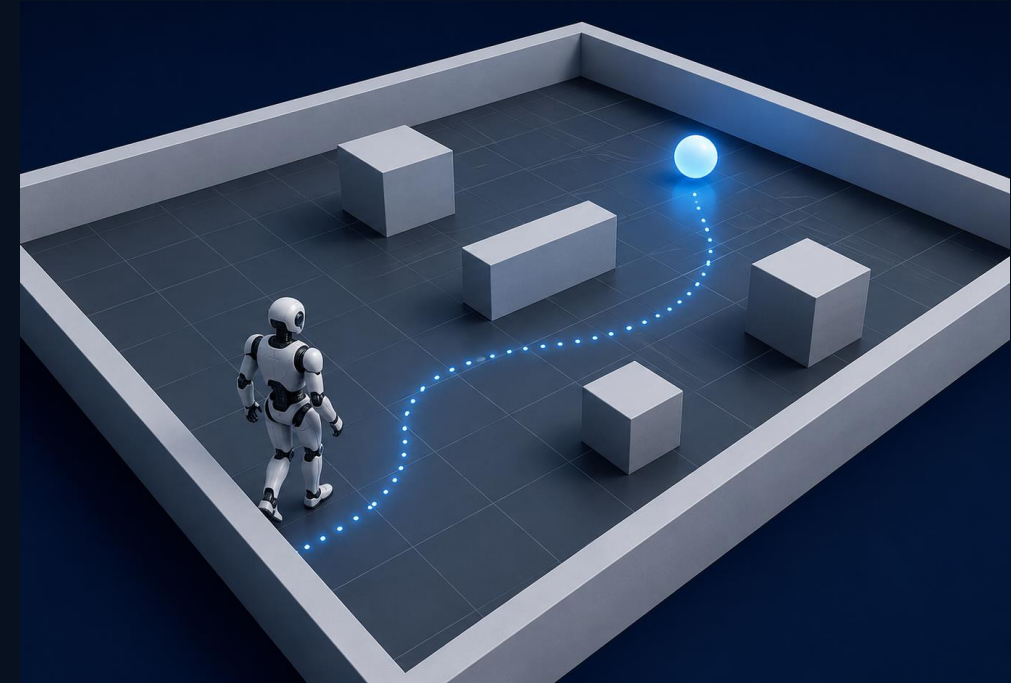
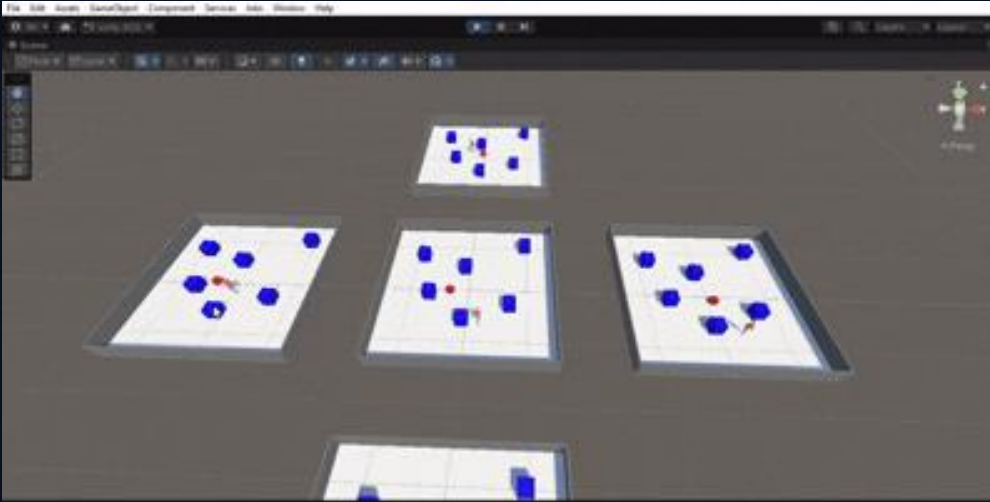


- Reality gap: physics, friction, motor delays and sensor noise
- Ray observations can map conceptually to LiDAR/depth readings
- Domain randomization can improve robustness
- Real deployment would require cautious testing and fine-tuning

Simulation makes experimentation safe and repeatable, but transfer requires handling imperfect real-world conditions.

Result Demo and Conclusion

Final project outcome



Completed outcome

- Unity robot navigation environment
- PPO + Behavioral Cloning + GAIL training
- RayPerceptionSensor3D LiDAR-style sensing
- Reward exploit identified and fixed
- TensorBoard result analysis

The project demonstrates a working reinforcement learning navigation prototype and a clear path for stronger future comparison experiments.